🏛️ TÀI LIỆU KIẾN TRÚC TỔNG THỂ HỆ THỐNG FINANCIAL MULTIMODAL RAG

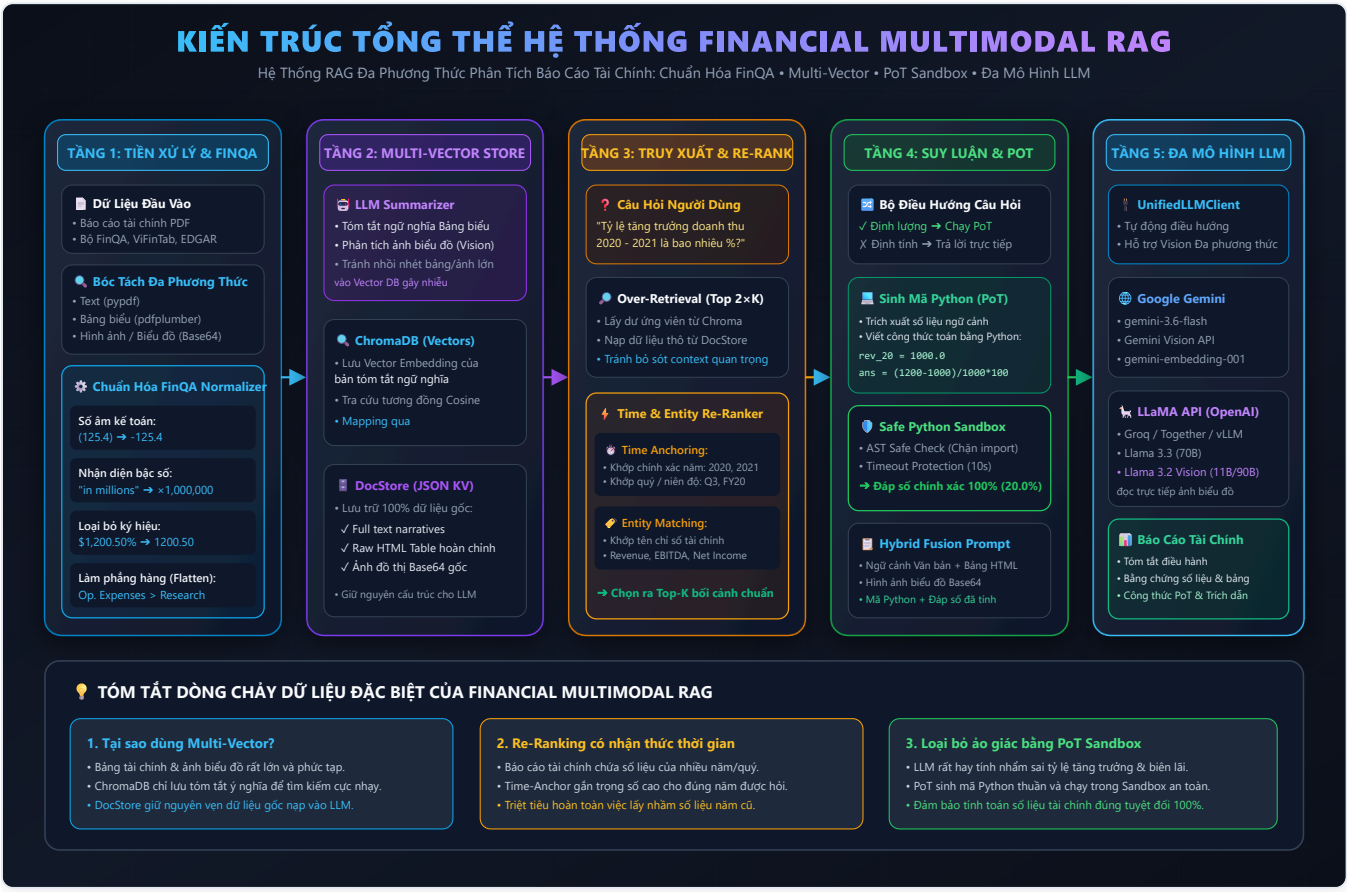
> ## Kiến Trúc Lai: Đồ Thị Tri Thức (Knowledge Graph) + Cây Phân Cấp Tài Liệu (Document Tree) + Multi-Vector

Tài liệu này mô tả chi tiết toàn bộ kiến trúc, lược đồ dữ liệu, luồng luân chuyển và các cơ chế kỹ thuật chuyên sâu được áp dụng trong hệ thống **Financial Multimodal RAG** (Hệ thống RAG Đa phương thức chuyên sâu cho phân tích báo cáo tài chính doanh nghiệp).

📸 1. Sơ Đồ Kiến Trúc Dạng Hình Ảnh (Visual Architecture Diagrams)

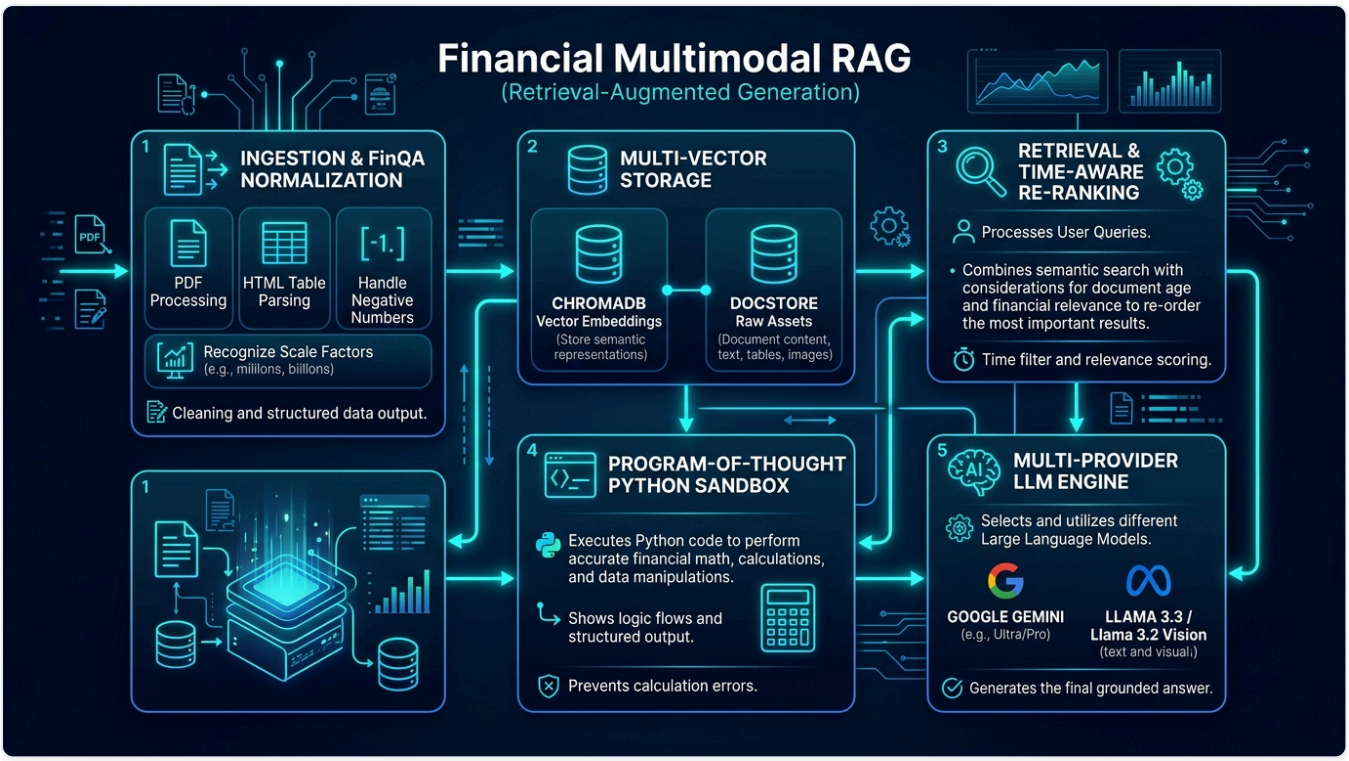
A. Sơ Đồ Vector Sắc Nét 100% Tiếng Việt (SVG Diagram)

> *Ghi chú: Hình ảnh vector bên dưới hiển thị sắc nét ở mọi độ phân giải (bạn có thể bấm vào để xem toàn màn hình).*



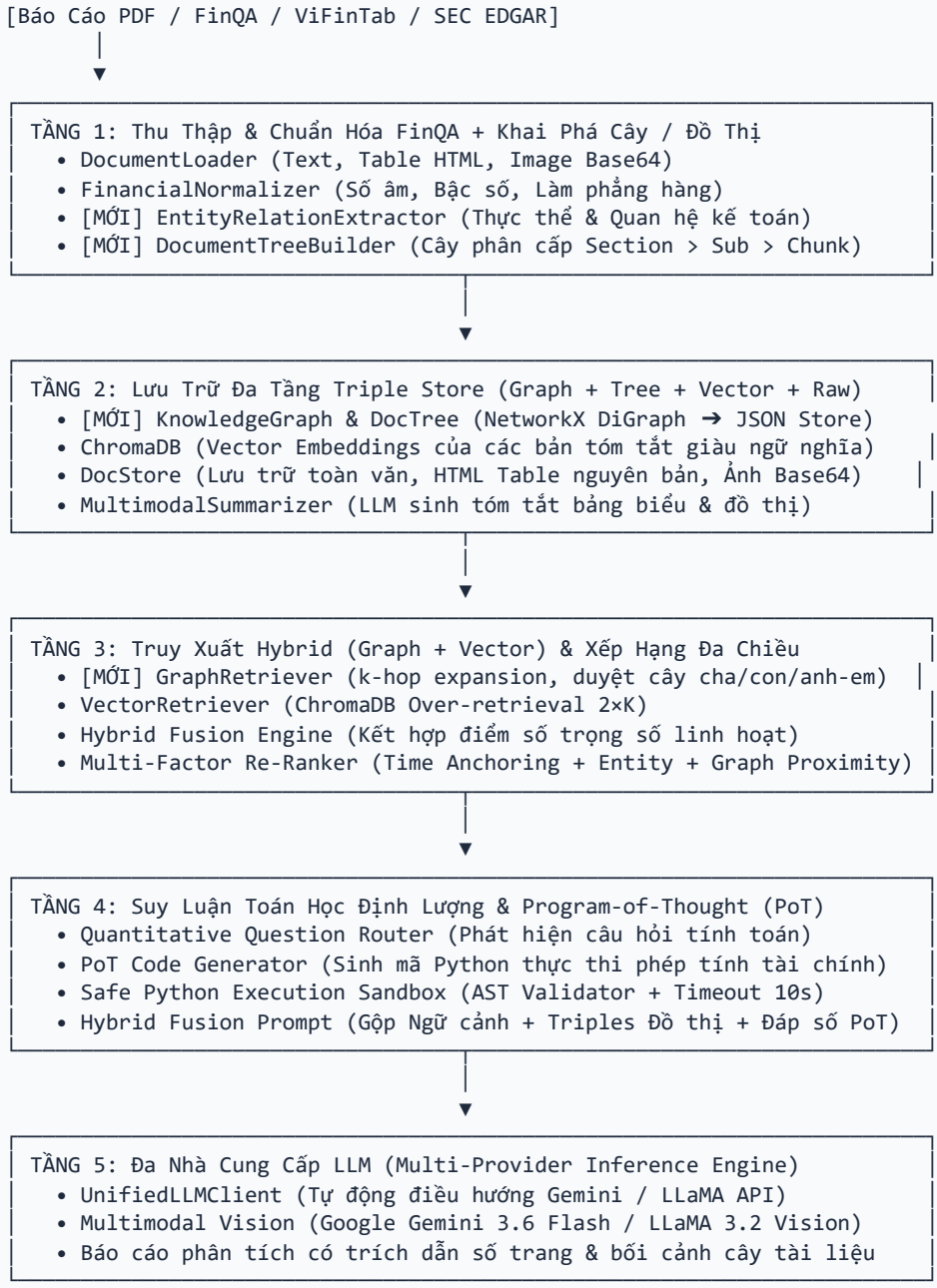
Sơ Đồ Kiến Trúc Tổng Thể Tiếng Việt

B. Sơ Đồ Đồ Họa Công Nghệ (Infographic Diagram)



Sơ Đồ Đồ Họa Công Nghệ Infographic

🔍 2. Phân Tích Chi Tiết 5 Tầng Kiến Trúc Hệ Thống



📦 TẦNG 1: Tiền Xử Lý, Chuẩn Hóa FinQA & Khai Phá Cây / Đồ Thị (Ingestion, Tree & Graph)

- **Module chịu trách nhiệm**: `document_loader.py`, `financial_normalizer.py`, `entity_extractor.py`, `tree_builder.py`, `models.py`.
- **Vấn đề giải quyết**:
  - Báo cáo tài chính chứa rất nhiều quy ước kế toán đặc thù khiến RAG thông thường thất bại (số âm trong ngoặc đơn, bậc số rải rác, tiêu đề bảng lồng nhau).
  - Quan trọng hơn, báo cáo tài chính là **tài liệu có cấu trúc phân cấp chặt chẽ** (Chương ➔ Mục ➔ Bảng ➔ Thuyết minh phụ trợ). Nếu chỉ cắt nhỏ tài liệu thành các đoạn văn độc lập (flat chunking), mối liên kết logic giữa các chỉ số tài chính và các thuyết minh liên quan sẽ bị đứt gãy hoàn toàn.

- **Cơ chế hoạt động**:
  1. **Bóc tách đa phương thức**:
     - **Văn bản**: Phân đoạn theo trang và trượt cửa sổ (sliding-window).
     - **Bảng biểu**: Dùng `pdfplumber` nhận diện lưới ô, giữ nguyên cấu trúc dòng, cột và chuyển sang HTML/Markdown.
     - **Đồ thị/Hình ảnh**: Trích xuất các biểu đồ biến động doanh thu, cơ cấu tài sản và lưu dạng ảnh Base64.
  2. **Chuẩn hóa kế toán FinQA (`FinancialNormalizer`)**:
     - Chuyển đổi số âm kế toán: `(125.4)` $\rightarrow$ `-125.4` .
     - Loại bỏ các ký tự gây nhiễu khi tính toán: `$` , `€` , `%` , dấu phẩy ngăn cách hàng nghìn.
     - Bóc tách quy mô tiền tệ: "*in millions*" $\rightarrow$ nhân 10^6, "*in thousands*" $\rightarrow$ nhân 10^3, "*tỷ đồng*" $\rightarrow$ nhân 10^9.
     - **Làm phẳng hàng tiêu đề nhóm (`flatten_row_headers`)**: Gán tiền tố nhóm cha vào từng mục con (ví dụ: `Operating Expenses > Research`).
  3. **Trích xuất Thực thể & Quan hệ Tài chính (`EntityRelationExtractor`)**:
     - Nhận diện các thực thể cốt lõi: Doanh nghiệp (`Company`), Chỉ số kế toán (`FinancialMetric`), Niên độ/Kỳ báo cáo (`TimePeriod`), Bảng biểu (`Table`), và Thuyết minh (`Footnote`).
     - Xác lập các mối quan hệ ngữ nghĩa: `CONTAINS` (Bảng chứa chỉ số), `DERIVED_FROM` (Chỉ số con cấu thành nên chỉ số tổng), `REPORTED_IN` (Chỉ số được báo cáo trong kỳ tài chính nào), `CROSS_REF` (Tham chiếu chéo tới Thuyết minh số X).
  4. **Xây dựng Cây Phân Cấp Tài Liệu (`DocumentTreeBuilder`)**:
     - Tự động nhận diện cấu trúc tiêu đề báo cáo dựa trên font chữ và mẫu ngữ pháp (Heading Levels).
     - Xây dựng cây phả hệ: `Root ➔ Section ➔ SubSection ➔ Chunk (Text / Table / Image)` .
     - Tự động sinh tóm tắt bối cảnh cho các Node Section cấp trên và liên kết con cháu qua `parent_id` .

💾 TẦNG 2: Lưu Trữ Đa Tầng Triple Store (Knowledge Graph + Vector + DocStore)

- **Module chịu trách nhiệm**: `knowledge_graph.py` , `vector_store.py` , `doc_store.py` , `multimodal_summarizer.py` .
- **Vấn đề giải quyết**:
  - Vector Database chỉ có khả năng so khớp ngữ nghĩa gần đúng (semantic similarity), hoàn toàn không hiểu được quan hệ kế toán đa chiều (ví dụ: *Lợi nhuận gộp = Doanh thu thuần - Giá vốn*).
  - Ngược lại, nếu chỉ dùng đồ thị thuần túy thì sẽ bỏ lỡ các thông tin định tính dạng văn bản tự do trong phần giải trình của ban điều hành (MD&A).
- **Cơ chế hoạt động**:
  1. **Đồ thị tri thức & Cây tài liệu (NetworkX DiGraph)**:
     - Lưu trữ toàn bộ các Node thực thể và Edges quan hệ dưới dạng đồ thị có hướng (Directed Graph).
     - Embed cây tài liệu ngay trong cùng một không gian đồ thị, cho phép vừa duyệt quan hệ thực thể tài chính vừa duyệt cấu trúc chương mục tài liệu.
     - Cơ chế bền vững: Tự động tuần tự hóa và lưu ra tệp `data/processed/knowledge_graph.json` theo định dạng chuẩn Node-Link JSON.
  2. **Vector Store (ChromaDB)**:

- Dùng LLM sinh bản tóm tắt súc tích, giàu từ khóa ngữ nghĩa cho từng bảng biểu và biểu đồ.
- Chỉ lưu Vector Embeddings của các bản **tóm tắt ngữ nghĩa** kèm định danh `doc_id` để tìm kiếm siêu tốc mà không bị loãng ngữ nghĩa.

3. **Document Store (DocStore JSON KV-Store)**:
   - Lưu trữ toàn văn bản gốc, mã bảng HTML đầy đủ, và chuỗi ảnh Base64 chất lượng cao.
   - Phục vụ truy xuất nội dung chi tiết đưa vào ngữ cảnh LLM khi có kết quả từ bước tìm kiếm.

🎯 TẦNG 3: Truy Xuất Kết Hợp Hybrid & Xếp Hạng Đa Chiều (Hybrid Retrieval & Re-ranking)

- **Module chịu trách nhiệm**: `graph_retriever.py`, `vector_store.py`, `reranker.py`.
- **Vấn đề giải quyết**: Một truy vấn tài chính như "*Biên lợi nhuận hoạt động quý 3/2021 thay đổi thế nào so với cùng kỳ?*" đòi hỏi kết hợp dữ liệu từ nhiều trang khác nhau (bảng P&L quý 3/2021, bảng P&L quý 3/2020, và phần phân tích biên lợi nhuận trong thuyết minh).
- **Cơ chế hoạt động**:
  1. **Truy xuất Đồ thị (Graph k-hop Traversal)**:
     - Trích xuất thực thể từ câu hỏi (ví dụ: `Operating Margin`, `Q3/2021`).
     - Định vị các node tương ứng trên đồ thị và thực hiện thuật toán mở rộng k-hop ($k = 2$) để thu thập toàn bộ các node lân cận liên quan: giá trị số, kỳ so sánh, và thuyết minh kèm theo.
  2. **Duyệt Cây Phân Cấp (Tree Breadcrumbs Context)**:
     - Từ các chunk được tìm thấy, đi ngược cây phả hệ để lấy tiêu đề chương lớn (Parent Section) và đi xuôi cây để lấy các bảng biểu/chỉ tiêu phụ trợ liên đới.
  3. **Truy xuất Ngữ nghĩa Vector (Over-Retrieval)**:
     - Lấy $2 \times K$ tài liệu ứng viên từ ChromaDB dựa trên độ tương đồng cosine của embedding câu hỏi.
  4. **Hợp nhất Hybrid (Hybrid Fusion Score)**: $$\text{Score}_{\text{final}} = \alpha \cdot \text{Score}_{\text{vector}} + \beta \cdot \text{Score}_{\text{graph}}$$ (Mặc định: $\alpha = 0.6, \beta = 0.4$, có thể tùy chỉnh linh hoạt qua `.env`).
  5. **Tái Xếp Hạng Đa Chiều (`ContextReRanker`)**:
     - **Time Anchoring**: Khớp chính xác năm tài chính (`2021`, `2022`), quý (`Q1`, `Q3`), hoặc niên độ (`FY2023`).
     - **Entity Matching**: Khớp tên chỉ số trọng yếu (*Revenue, Gross Profit, Operating Expense, EBITDA*).
     - **Graph Proximity Score**: Ưu tiên các đoạn có liên kết trực tiếp trên đồ thị tri thức với thực thể câu hỏi.
     - **Tree Depth Score**: Ưu tiên các dữ liệu nằm trong cùng một Section chuyên mục của báo cáo.

🧠 TẦNG 4: Suy Luận Định Lượng & Program-of-Thought (PoT Layer)

- **Module chịu trách nhiệm**: `pot_reasoning.py`, `sandbox.py`, `financial_qa.py`.
- **Vấn đề giải quyết**: LLM không đáng tin cậy khi thực hiện các phép tính tài chính phức tạp (ảo giác số học, tính nhẩm sai số phần trăm, cộng trừ sai hàng đơn vị).
- **Cơ chế hoạt động**:
  1. **Bộ phân loại câu hỏi (Quantitative Router)**:

- Phát hiện câu hỏi định lượng số học thông qua từ khóa nhận diện (*"tỷ lệ", "bao nhiêu", "tăng trưởng", "tính", "growth rate", "percentage", "margin"*).

2. **Sinh mã tính toán (PoT Generator)**:
   - Chỉ định LLM viết một đoạn mã Python chuẩn mực trích xuất các biến số từ ngữ cảnh và thực hiện phép tính, gán kết quả vào biến `ans`.
3. **Chạy trong Safe Python Execution Sandbox**:
   - **AST Validation**: Quét cây cú pháp trừu tượng, chặn đứng hoàn toàn các lệnh nguy hiểm (`import`, `open`, `eval`, `exec`, `subprocess`).
   - **Timeout Protection**: Giới hạn tối đa 10 giây để chống lặp vô hạn.
   - **Built-in an toàn**: Chỉ cho phép sử dụng các hàm số học cơ bản (`round`, `abs`, `min`, `max`, `sum`).
   - Kết quả tính toán đạt độ chính xác 100%.
4. **Hybrid Context Fusion**:
   - Gộp kết quả tính toán cùng các Subgraph Triples từ đồ thị tri thức và dữ liệu bảng HTML vào prompt cuối cùng để LLM diễn giải phân tích.

🚀 TẦNG 5: Đa Nhà Cung Cấp LLM (Multi-Provider Inference Engine)

- **Module chịu trách nhiệm**: `llm_client.py`, `financial_qa.py`.
- **Vấn đề giải quyết**: Tránh phụ thuộc độc quyền vào một nhà cung cấp điện toán đám mây duy nhất, linh hoạt tối ưu hóa chi phí và hiệu năng.
- **Cơ chế hoạt động**:
  1. **Lớp trừu tượng** `UnifiedLLMClient`:
     - Nhận diện `LLM_PROVIDER` cấu hình trong `.env` để tự động điều phối gọi Google GenAI SDK hoặc OpenAI-compatible REST API (Groq, Together AI, OpenRouter, vLLM, Ollama).
  2. **Hỗ trợ Multimodal Vision**:
     - **Google Gemini**: Đóng gói ảnh bằng `types.Part.from_bytes` để đọc trực tiếp đồ thị.
     - **LLaMA Vision** (`llama-3.2-11b-vision-preview` / `llama-3.2-90b-vision-preview`): Đóng gói ảnh dưới dạng Base64 Data URL.
  3. **Báo cáo phân tích chuyên sâu**:
     - Tóm tắt điều hành (Executive Summary).
     - Phân tích chi tiết số liệu từ bảng biểu và đồ thị.
     - Mạng lưới thực thể và quan hệ từ Đồ thị tri thức (Knowledge Graph Triples).
     - Công thức tính toán và đáp số được Sandbox kiểm chứng.
     - Trích dẫn rõ ràng đường dẫn cây tài liệu (Document Breadcrumb) và số trang nguồn.

---

3. Lược Đồ Đồ Thị Tri Thức & Cây Tài Liệu (Knowledge Graph Schema)

A. Các Loại Node (Entity Types)

Loại Node (`type`)	Thuộc Tính Cốt Lõi (`attributes`)	Mục Đích Sử Dụng
`company`	`name` , `ticker` , `cik` , `industry`	Định danh doanh nghiệp phát hành báo cáo
`metric`	`name` , `normalized_name` , `category` , `unit`	Chỉ tiêu tài chính (Doanh thu, Lợi nhuận gộp, EBITDA...)
`period`	`year` , `quarter` , `fiscal_year` , `period_type`	Mốc thời gian báo cáo (FY2021, Q3/2022)
`value`	`raw_value` , `numeric_value` , `scale` , `currency`	Con số cụ thể đã được chuẩn hóa
`section`	`title` , `level` , `page_start` , `page_end`	Chương / Mục lớn trong tài liệu (Document Tree Node)
`subsection`	`title` , `parent_section_id` , `page`	Tiểu mục con trong chương
`table`	`table_id` , `headers` , `row_count` , `col_count`	Bảng biểu tài chính
`chart`	`image_id` , `chart_type` , `summary`	Biểu đồ, đồ thị biến động số liệu
`chunk`	`chunk_id` , `modality` , `page` , `source_file`	Đơn vị văn bản nhỏ nhất được lập chỉ mục vector

B. Các Loại Cạnh Quan Hệ (Relation / Edge Types)

Mối Quan Hệ (`relation_type`)	Node Nguồn ➔ Node Đích	Ý Nghĩa Kế Toán / Cấu Trúc
`CONTAINS`	`section` ➔ `subsection` / `chunk`	Cấu trúc phân cấp cha - con của cây tài liệu
`REPORTED_IN`	`metric` ➔ `period`	Chỉ số tài chính được ghi nhận trong kỳ nào
`HAS_VALUE`	`metric` ➔ `value`	Giá trị cụ thể của chỉ số trong một ngữ cảnh
`DERIVED_FROM`	`metric` ➔ `metric`	Quan hệ cấu thành (ví dụ: *Gross Profit* hình thành từ *Revenue* và *COGS*)
`CROSS_REF`	`table` ➔ `section` (Footnote)	Tham chiếu chéo giữa bảng số liệu và Thuyết minh giải trình
`MENTIONS`	`chunk` ➔ `metric` / `company`	Đoạn văn bản đề cập đến một thực thể tài chính

---

📊 4. Bảng So Sánh Các Mô Hình Hỗ Trợ

Tiêu Chí	Google Gemini (`gemini-3.6-flash`)	LLaMA 3.3 70B (Groq / OpenRouter)	LLaMA 3.2 Vision (11B / 90B)
Kiến trúc dữ liệu	Multimodal (Text + Image + Table)	Ngôn ngữ thuần (Text & Table)	Multimodal (Text + Vision)
Đọc biểu đồ/ảnh	Rất tốt (Native Vision)	Không (chỉ nhận text/table)	Rất tốt (Đọc biểu đồ qua Base64)
Khả năng sinh code PoT	Xuất sắc	Rất mạnh & Chuẩn xác	Rất tốt
Giao thức kết nối	Google GenAI SDK	OpenAI-compatible REST API	OpenAI-compatible REST API
Tốc độ suy luận	Nhanh (~80 tok/s)	Cực nhanh qua Groq (~300 tok/s)	Nhanh qua Groq / OpenRouter

---

🛠️ 5. Cấu Hình Biến Môi Trường (.env) Cho Hệ Thống Mới

ENV

```
# ================================================================================
# 1. CẤU HÌNH ĐỒ THỊ TRI THỨC & CÂY PHÂN CẤP TÀI LIỆU (KNOWLEDGE GRAPH)
# ================================================================================
GRAPH_STORE_PATH=data/processed/knowledge_graph.json
ENABLE_GRAPH_EXTRACTION=true
GRAPH_RETRIEVAL_HOPS=2
GRAPH_WEIGHT=0.4
VECTOR_WEIGHT=0.6

# ================================================================================
# 2. CHỌN NHÀ CUNG CẤP LLM (gemini HOẶC llama)
# ================================================================================
LLM_PROVIDER=gemini

# Cấu hình Google Gemini
GEMINI_API_KEY=your_gemini_api_key_here
EMBEDDING_MODEL=gemini-embedding-001
CHAT_MODEL=gemini-3.6-flash

# Cấu hình LLaMA qua Groq (nếu dùng LLM_PROVIDER=llama)
LLAMA_BASE_URL=https://api.groq.com/openai/v1
LLAMA_API_KEY=gsk_your_groq_api_key_here
LLAMA_CHAT_MODEL=llama-3.3-70b-versatile
```